

UNIVERSITÉ DE YAOUNDÉ I
FACULTÉ DES SCIENCES
DÉPARTEMENT D'INFORMATIQUE
LICENCE 3 INFORMATIQUE
ANNÉE ACADÉMIQUE 2025 – 2026

RAPPORT DE TRAVAUX PRATIQUES
Traitement d'Images Numériques en Langage C

INF3722 : Traitement D'images

Nom et Prénom : *KAMDOM NGUETCHESSI MERVEILLE PRISCA*
Matricule : *23V2060*

Sous la supervision de : *Dr. TAPAMO KENFACK Hippolyte Michel*

1. Introduction

Ce rapport présente le travail réalisé dans le cadre du travail pratique de traitement d'images numériques, un domaine qui occupe une place centrale dans la formation en informatique et en science des données. Ce TP nous a permis de mettre en pratique, sous la forme d'un programme complet écrit en langage C, l'ensemble des notions théoriques abordées durant l'année : de la lecture d'un fichier image jusqu'aux traitements les plus avancés que sont la transformée de Fourier, la segmentation et la morphologie mathématique.

Plutôt que de traiter chaque chapitre du cours de manière isolée, nous avons choisi de construire un unique programme modulaire et interactif, capable d'appliquer, sur une image réelle comme sur une image de synthèse, l'ensemble des algorithmes étudiés, et de ranger automatiquement les résultats obtenus de façon claire et exploitable.

2. Objectifs pédagogiques

Au-delà de la simple implémentation des algorithmes, ce travail poursuivait plusieurs objectifs complémentaires. Le premier était de consolider notre compréhension des fondements mathématiques du traitement d'images (convolution, transformée de Fourier, morphologie, segmentation) en les traduisant en code fonctionnel, sans recourir à des bibliothèques externes de traitement d'images.

Le second objectif était de développer une rigueur de programmation en langage C : gestion manuelle de la mémoire, structuration du code en modules cohérents, séparation entre la logique de calcul et l'interface utilisateur, et respect des bonnes pratiques de compilation (compilation sans avertissement avec les options `-Wall -Wextra`).

Enfin, un dernier objectif, plus applicatif, était de proposer une expérience utilisateur soignée autour de ces algorithmes : un menu interactif permettant de choisir librement l'image, le cours et le groupe d'opérations à exécuter, ainsi qu'un classement rigoureux des résultats produits.

3. Architecture générale du programme

Le programme est structuré selon le principe de séparation des responsabilités : chaque fichier source correspond à une unité thématique précise du cours, ce qui facilite la lecture, la maintenance et l'évolution du code.

Les fichiers du projet se répartissent ainsi :

- `image.h` : structures de données centrales (`ImageGris`, `ImageCouleur`), macros d'accès aux pixels et déclarations de toutes les fonctions du projet.

- `io_image.c` : lecture et écriture des formats PGM (P5) et PPM (P6), allocation et conversion entre images en niveaux de gris et en couleur.
- `traitement_base.c` : histogramme, statistiques, transformations de contraste, égalisation et interpolation (Cours 02).
- `convolution.c` : filtrage spatial linéaire et non linéaire (Cours 03).
- `fourier.c` : transformée de Fourier rapide et filtrage fréquentiel (Cours 04).
- `contours.c` : opérateurs de détection de contours (Cours 05).
- `segmentation.c` : méthodes de segmentation d'image (Cours 06).
- `images_binaires.c` : morphologie mathématique et analyse topologique (Cours 07).
- `main.c` : interface utilisateur interactive, orchestration des traitements et organisation des sorties.

Cette organisation garantit qu'aucune fonction de calcul n'a été modifiée lors de l'ajout de l'interface interactive : `main.c` se contente d'appeler les fonctions déjà existantes et de mettre en forme leurs résultats, sans jamais altérer la logique algorithmique développée au fil des différents cours.

4. Fonctionnalités implémentées par module

Le tableau ci-dessous récapitule, cours par cours, les principales fonctionnalités mises en œuvre dans le programme.

Module	Thème du cours	Contenu principal implémenté
Cours 02	Traitements de base	Histogramme, statistiques (luminance, contraste), transformations de contraste (linéaire, saturation, gamma, seuillage), égalisation globale et locale, opérations arithmétiques et logiques entre images, interpolation (plus proche voisin, bilinéaire, bicubique).
Cours 03	Convolution	Convolution 2D générique avec gestion des bords (miroir), filtres moyenneur, gaussien, médian, min et max.
Cours 04	Transformée de Fourier	FFT 1D (Cooley-Tukey, radix-2) étendue en FFT 2D, recentrage du spectre (fftshift), filtres fréquentiels passe-bas, passe-haut et coupe-bande (Notch), rehaussement de contraste fréquentiel.
Cours 05	Détection de contours	Dérivées discrètes, opérateurs de Roberts, Prewitt, Sobel et isotropique, laplaciens (4 et 8 connexités), laplacien de gaussienne (LoG) et détection des passages par zéro.
Cours 06	Segmentation	Seuillage global, multi-seuillage, seuillage d'Otsu, seuillage adaptatif, classification par K-moyennes, division-fusion par quadtree et croissance de régions (mono et multi-germes).
Cours 07	Images binaires	Binarisation, connexités 4 et 8, cartes de distances discrètes (D4, D8), codage de Freeman, étiquetage de composantes connexes, morphologie mathématique binaire et en niveaux de gris.

5. Interface utilisateur et organisation des résultats

L'un des apports principaux de ce travail réside dans la conception d'une interface en mode texte, entièrement interactive, qui guide l'utilisateur à travers les différentes possibilités du programme plutôt que d'exécuter tous les traitements en bloc. Au lancement, l'utilisateur choisit d'abord de travailler sur une image importée (tank.pgm ou sailboat.ppm) ou sur une texture périodique générée automatiquement, puis sélectionne le cours à appliquer et, enfin, le groupe d'opérations précis qu'il souhaite exécuter.

Cette texture périodique synthétique mérite une mention particulière : générée par la combinaison de deux fonctions sinusoïdales, elle produit un spectre de Fourier présentant des pics nets et localisés, ce qui en fait un support idéal pour valider visuellement l'action du filtre coupe-bande (Notch) et, plus largement, pour observer clairement l'effet des filtres fréquentiels sur une image dont le contenu est parfaitement maîtrisé.

Pour chaque groupe d'opérations exécuté, le programme affiche dans le terminal les statistiques essentielles de chaque résultat (valeurs minimale, maximale et moyenne des pixels), avant de les sauvegarder. Les fichiers générés sont ensuite rangés automatiquement dans l'un des sept dossiers suivants, chacun dédié à une catégorie précise :

Dossier	Contenu
00_images_entree/	Copie des deux images d'entrée (tank.pgm, sailboat.ppm) et de la texture périodique synthétique générée pour la démonstration du filtre de Fourier.
cours02_traitements_base/	Résultats des traitements de base : histogramme, profil d'intensité, contraste, interpolation.
cours03_convolution/	Résultats des filtres de convolution (moyenneur, gaussien, médian, min, max).
cours04_fourier/	Spectres, filtres fréquentiels et résultat du filtre coupe-bande sur la texture synthétique.
cours05_contours/	Résultats des opérateurs de détection de contours.
cours06_segmentation/	Résultats des différentes méthodes de segmentation.
cours07_images_binaires/	Résultats des traitements morphologiques et de l'analyse topologique.

Ce classement permet de retrouver immédiatement, pour un cours donné, l'ensemble des images produites, sans avoir à parcourir un unique dossier contenant plusieurs centaines de fichiers mélangés.

6. Un point technique particulier : la lisibilité de l'histogramme

Dans une première version du programme, l'histogramme et le profil d'intensité d'une image étaient représentés uniquement par un affichage textuel dans le terminal, sous forme de barres composées de caractères ASCII. Si cette solution avait le mérite de la simplicité, elle restait peu lisible et rendait difficile toute analyse fine de la répartition des niveaux de gris.

Cette limite a été corrigée : seule la méthode de restitution a été revue. Le programme construit désormais une véritable image en niveaux de gris, au format PGM, dans laquelle chaque colonne représente une barre proportionnelle à l'effectif du niveau de gris correspondant pour l'histogramme, et une courbe continue pour le profil d'intensité extrait le long d'une ligne de l'image. Ces deux fichiers sont sauvegardés automatiquement dans le dossier du Cours 02, et peuvent être ouverts comme n'importe quelle photographie pour une lecture visuelle immédiate et beaucoup plus parlante que la représentation textuelle initiale.

7. Compilation et exécution

Le projet est fourni avec un Makefile qui automatise l'ensemble de la compilation. La commande `make` compile tous les modules et produit l'exécutable `traitement_image`, sans avertissement du compilateur, les options `-Wall -Wextra` ayant été activées tout au long du développement afin de garantir la propreté du code. Les images d'entrée (`tank.pgm` et `sailboat.ppm`) doivent être placées dans le dossier `images_source/` avant l'exécution ; le programme crée alors automatiquement l'arborescence `resultats/` décrite plus haut.

La commande `make clean` ne supprime que les fichiers de compilation (objets et exécutable) et préserve systématiquement les images d'entrée ainsi que les résultats déjà générés, afin d'éviter toute perte accidentelle de données lors d'une recompilation.

8. Difficultés rencontrées et apprentissages

La principale difficulté de ce travail a résidé dans la gestion rigoureuse de la mémoire dynamique : chaque image allouée devait être libérée au moment opportun, en particulier dans les fonctions qui manipulent plusieurs images intermédiaires, comme le traitement couleur canal par canal ou les opérations entre paires d'images. La mise en place d'une structure de « lot » de résultats, centralisant l'affichage, la sauvegarde et la libération mémoire, a permis d'unifier ce traitement et de réduire sensiblement les risques de fuite mémoire.

La transformée de Fourier a également constitué un exercice exigeant, tant sur le plan mathématique (implémentation récursive de l'algorithme de Cooley-Tukey, gestion des nombres complexes, recentrage du spectre) que sur le plan de la validation, puisqu'il fallait s'assurer que la

reconstruction par FFT inverse redonnait bien l'image d'origine avant de pouvoir faire confiance aux filtres fréquentiels construits par la suite.

9. Conclusion

Ce travail pratique nous a permis de consolider, par la pratique, l'ensemble des notions de traitement d'images abordées durant l'année : filtrage spatial et fréquentiel, détection de contours, segmentation et morphologie mathématique. Au-delà de l'implémentation des algorithmes eux-mêmes, la réalisation d'une interface interactive et d'une organisation claire des résultats nous a sensibilisés à l'importance de l'expérience utilisateur et de la lisibilité, y compris dans un projet à vocation académique. Ce projet constitue ainsi une base solide, aisément extensible, sur laquelle pourraient venir s'appuyer des travaux futurs.

Compilation et execution : `make && ./traitement_image` dans le terminal